Name: Soham Chavan

PRN: 124B1F022

Class: IT A

Assignment No. 6

Aim:

To model the disaster relief truck loading problem as a 0/1 Knapsack problem, find the set of items that maximizes total utility within the truck's weight limit using Dynamic Programming, compare it with Brute Force and Greedy approaches, and extend the model to give priority to perishable or life-saving items and to use multiple trucks.

Theory:

Problem Overview

The 0/1 Knapsack Problem is a classic optimization problem in which a subset of items must be selected to maximize the total value without exceeding a given weight limit. Each item i has a weight[i] and a value (utility) value[i], and the truck has capacity W. Each item is either taken (1) or not taken (0).

Objective: Maximize the sum of value[i] * x[i], subject to the sum of weight[i] * x[i] <= W, where x[i] is 0 or 1.

Algorithmic Approaches

1. Brute Force: generate all possible subsets of items, calculate the total weight and value of each, and select the subset with the highest value whose weight is within W. Time complexity: O(2^n).

2. Dynamic Programming: uses overlapping subproblems and optimal substructure. A table dp[i][w] stores the maximum value using the first i items with capacity w. Time complexity: O(N * W); space complexity: O(N * W).

```
if weight[i-1] <= w:
    dp[i][w] = max(value[i-1] + dp[i-1][w - weight[i-1]], dp[i-1][w])
else:
    dp[i][w] = dp[i-1][w]
```

3. Greedy Approach (for Fractional Knapsack): sort the items by value/weight ratio and pick the highest ratio items first until the capacity is filled. For the 0/1 version this may give a suboptimal answer.

Working of the Algorithm

1. Input: N items with weight w[i] and utility v[i], and truck capacity W.
2. Initialize the DP table dp[N+1][W+1] to 0.
3. For each item i from 1 to N and each capacity w from 1 to W, fill the table using the recurrence above.
4. The maximum utility is dp[N][W].
5. Trace back to find the included items: start from dp[N][W]; for i = N down to 1, if dp[i][w] is not equal to dp[i-1][w], include item i and update w = w - weight[i-1].
6. Assign priority to perishable or life-saving items by increasing their utility where needed.

Extensions

- Perishable items (medicines, food) get a higher utility so that they are preferred.
- Multiple trucks: load one truck optimally, remove its items, and repeat for the next truck.
- Real-time decisions: re-run the DP when new items or capacity changes arrive.

Code:

```
import java.util.*;

public class KnapsackRelief {

    static String[] names = {"Medical Kit", "Drinking Water", "Food Packets", "Baby
Formula",
            "Blankets", "Tent", "Torch Set", "Power Tools"};
    static int[] weight = {8, 15, 12, 5, 10, 20, 3, 18};
    static int[] utility = {90, 70, 60, 45, 40, 50, 20, 30};
    static boolean[] critical = {true, true, true, true, false, false, false, false};

    // ---------- 0/1 Knapsack using Dynamic Programming ----------
    static int dpKnapsack(int[] w, int[] v, int W, List<Integer> chosen) {
        int n = w.length;
        int[][] dp = new int[n + 1][W + 1];
        for (int i = 1; i <= n; i++) {
            for (int c = 1; c <= W; c++) {
                if (w[i - 1] <= c)
                    dp[i][c] = Math.max(v[i - 1] + dp[i - 1][c - w[i - 1]], dp[i - 1]
[c]);
                else
                    dp[i][c] = dp[i - 1][c];
            }
        }
        int c = W;
        for (int i = n; i >= 1; i--) {
            if (dp[i][c] != dp[i - 1][c]) {
                chosen.add(i - 1);
                c -= w[i - 1];
            }
        }
        Collections.reverse(chosen);
        return dp[n][W];
    }

    // ---------- Brute Force: try all 2^n subsets ----------
    static int bruteForce(int[] w, int[] v, int W, List<Integer> chosen) {
        int n = w.length, best = 0, bestMask = 0;
        for (int mask = 0; mask < (1 << n); mask++) {
            int tw = 0, tv = 0;
            for (int i = 0; i < n; i++)
                if ((mask & (1 << i)) != 0) { tw += w[i]; tv += v[i]; }
            if (tw <= W && tv > best) { best = tv; bestMask = mask; }
        }
        for (int i = 0; i < n; i++) if ((bestMask & (1 << i)) != 0) chosen.add(i);
        return best;
```

```
    }

    // ---------- Greedy: sort by value/weight ratio, take whatever fits ----------
    static int greedy(int[] w, int[] v, int W, List<Integer> chosen) {
        Integer[] idx = new Integer[w.length];
        for (int i = 0; i < idx.length; i++) idx[i] = i;
        Arrays.sort(idx, (a, b) -> Double.compare((double) v[b] / w[b], (double)
v[a] / w[a]));
        int used = 0, total = 0;
        for (int i : idx) {
            if (used + w[i] <= W) { used += w[i]; total += v[i]; chosen.add(i); }
        }
        Collections.sort(chosen);
        return total;
    }

    static String describe(List<Integer> chosen) {
        StringBuilder sb = new StringBuilder();
        int tw = 0;
        for (int i : chosen) {
            if (sb.length() > 0) sb.append(", ");
            sb.append(names[i]);
            tw += weight[i];
        }
        return sb + "\n  Total weight used: " + tw + " kg";
    }

    public static void main(String[] args) {
        int W = 50;
        int n = names.length;

        System.out.println("Truck capacity: " + W + " kg\n");
        System.out.printf("%-16s %8s %8s %10s%n", "Item", "Weight", "Utility",
"Critical");
        for (int i = 0; i < n; i++)
            System.out.printf("%-16s %8d %8d %10s%n", names[i], weight[i], utility[i],
critical[i] ? "Yes" : "No");

        System.out.println("\n=== Comparison of approaches (base utilities) ===");
        List<Integer> c1 = new ArrayList<>(), c2 = new ArrayList<>(), c3 = new
ArrayList<>();
        int dp = dpKnapsack(weight, utility, W, c1);
        int bf = bruteForce(weight, utility, W, c2);
        int gr = greedy(weight, utility, W, c3);
        System.out.println("Dynamic Programming : utility = " + dp + "\n  Items: " +
describe(c1));
        System.out.println("Brute Force         : utility = " + bf + "\n  Items: " +
describe(c2));
        System.out.println("Greedy (ratio)      : utility = " + gr + "\n  Items: " +
describe(c3));

        System.out.println("\nWork done for N = " + n + ", W = " + W + ":");
        System.out.println("  Brute Force subsets checked : " + (1L << n) + "
(O(2^N))");
```

```
        System.out.println("  DP table cells computed     : " + (n * (long) W) + "
(O(N*W))");
        int bigN = 30, bigW = 1000;
        System.out.println("For N = " + bigN + ", W = " + bigW + ":");
        System.out.println("  Brute Force subsets : " + (1L << bigN));
        System.out.println("  DP table cells      : " + (bigN * (long) bigW));

        System.out.println("\n=== Priority for perishable / life-saving items (utility
x 1.5) ===");
        int[] boosted = new int[n];
        for (int i = 0; i < n; i++) boosted[i] = critical[i] ? utility[i] + utility[i]
/ 2 : utility[i];
        List<Integer> c4 = new ArrayList<>();
        int pr = dpKnapsack(weight, boosted, W, c4);
        System.out.println("DP with priority    : utility = " + pr + "\n  Items: " +
describe(c4));

        System.out.println("\n=== Extension: multiple trucks (each " + W + " kg,
loaded one after another) ===");
        boolean[] taken = new boolean[n];
        for (int truck = 1; truck <= 2; truck++) {
            List<Integer> avail = new ArrayList<>();
            for (int i = 0; i < n; i++) if (!taken[i]) avail.add(i);
            if (avail.isEmpty()) break;
            int[] w2 = new int[avail.size()], v2 = new int[avail.size()];
            for (int k = 0; k < avail.size(); k++) {
                w2[k] = weight[avail.get(k)];
                v2[k] = boosted[avail.get(k)];
            }
            List<Integer> local = new ArrayList<>();
            int val = dpKnapsack(w2, v2, W, local);
            List<Integer> real = new ArrayList<>();
            for (int k : local) { real.add(avail.get(k)); taken[avail.get(k)] =
true; }
            System.out.println("Truck " + truck + ": utility = " + val + "\n  Items: "
+ describe(real));
        }
        StringBuilder left = new StringBuilder();
        for (int i = 0; i < n; i++) if (!taken[i]) left.append(left.length() > 0 ? ",
" : "").append(names[i]);
        System.out.println("Left behind: " + (left.length() == 0 ? "None" : left));
    }
}
```

Output:

```
Truck capacity: 50 kg

Item               Weight  Utility   Critical
Medical Kit             8       90        Yes
Drinking Water         15       70        Yes
Food Packets           12       60        Yes
Baby Formula            5       45        Yes
Blankets               10       40         No
```

```
Tent                   20       50         No
Torch Set               3       20         No
Power Tools            18       30         No

=== Comparison of approaches (base utilities) ===
Dynamic Programming : utility = 305
  Items: Medical Kit, Drinking Water, Food Packets, Baby Formula, Blankets
  Total weight used: 50 kg
Brute Force         : utility = 305
  Items: Medical Kit, Drinking Water, Food Packets, Baby Formula, Blankets
  Total weight used: 50 kg
Greedy (ratio)      : utility = 285
  Items: Medical Kit, Drinking Water, Food Packets, Baby Formula, Torch Set
  Total weight used: 43 kg

Work done for N = 8, W = 50:
  Brute Force subsets checked : 256  (O(2^N))
  DP table cells computed     : 400  (O(N*W))
For N = 30, W = 1000:
  Brute Force subsets : 1073741824
  DP table cells      : 30000

=== Priority for perishable / life-saving items (utility x 1.5) ===
DP with priority    : utility = 437
  Items: Medical Kit, Drinking Water, Food Packets, Baby Formula, Blankets
  Total weight used: 50 kg

=== Extension: multiple trucks (each 50 kg, loaded one after another) ===
Truck 1: utility = 437
  Items: Medical Kit, Drinking Water, Food Packets, Baby Formula, Blankets
  Total weight used: 50 kg
Truck 2: utility = 100
  Items: Tent, Torch Set, Power Tools
  Total weight used: 41 kg
Left behind: None
```

Conclusion:

The Dynamic Programming solution and the Brute Force solution both found the optimal utility of 305 for a 50 kg truck, while the Greedy ratio method reached only 285, showing that greedy selection does not guarantee an optimal 0/1 Knapsack solution. Brute Force checks 2^N subsets (over one billion for 30 items), whereas the DP table needs only N x W cells (30,000 for N = 30 and W = 1000), so Dynamic Programming is the practical choice. Increasing the utility of critical items (medicines, water, food, baby formula) by 50% kept them in the truck, and loading a second truck with the remaining items ensured that nothing was left behind. Hence the model efficiently allocates limited capacity to maximize relief utility and can be adapted for multiple trucks and dynamic supply chain decisions.